

システムソフトウェア・試験問題

2025 年度 (2025 年 11 月 27 日・試験時間 100 分)

書籍, 配布資料およびノート等は参照してはならない.
ただし, 最大一枚までのメモ (手書きに限る. A4 両面使用可) を参照できるものとする.

1. xv6¹のカーネルを構成する関数を用いてセマフォを実装し (コード 1), それを用いて生産者-消費者問題の解を作成した (コード 2). 関数 `setup` を実行した後, 関数 `producer` および `consumer` をそれぞれ別のカーネルスレッド (以下スレッド) で実行するものとする. また N は正の整数とする.

構造体 `spinlock` および関数 `acquire`, `release` は xv6 におけるスピンのロックの実装である. 関数 `sleep` および `wakeup` の定義をコード 3 に示す.

```
1 struct semaphore {
2     struct spinlock lock;
3     int count;
4 };
5
6 void P(struct semaphore *s) {
7     acquire(&s->lock);
8     while (s->count == 0) {
9         sleep(s, &s->lock);
10    }
11    s->count -= 1;
12    release(&s->lock);
13 }
14
15 void V(struct semaphore *s) {
16     acquire(&s->lock);
17     s->count += 1;
18     wakeup(s);
19     release(&s->lock);
20 }
```

コード 1: セマフォの定義

(a) コード 2 の空欄 `A` ~ `H` に入る式をそれぞれ答えよ.

```
1 struct semaphore ne, nf;
2 struct spinlock mtxp, mtxc;
3 data_t queue[N];
4 uint64 ip = 0, ic = 0;
5
6 void setup() {
7     initlock(&ne.lock, "ne");
8     ne.count = 0;
9     initlock(&nf.lock, "nf");
10    nf.count = N; // Nは正の整数
11    initlock(&mtxp, "mtxp");
12    initlock(&mtxc, "mtxc");
13 }
14
15 void producer() {
16     for (;;) {
17         data_t x;
18         x = produce_datum();
19         assert(A);
20         P(B);
21         acquire(C);
22         queue[ip++ % N] = x;
23         release(D);
24         V(E);
25     }
26 }
27
28 void consumer() {
29     for (;;) {
30         data_t x;
31         P(F);
32         acquire(G);
33         x = queue[ic++ % N];
34         release(H);
35         V(I);
36         assert(J);
37         consume_datum(x);
38     }
39 }
```

コード 2: セマフォによる生産者・消費者問題の解

¹以下, すべて RISC-V 版 xv6 を指す.

```

1 void sleep(void *chan, struct spinlock *lk) {
2     struct proc *p = myproc();
3     acquire(&p->lock);
4     release(lk);
5     p->chan = chan;
6     p->state = SLEEPING;
7     sched();
8     p->chan = 0;
9     release(&p->lock);
10    acquire(lk);
11 }
12
13 void wakeup(void *chan) {
14     struct proc *p;
15     for (p = proc; p < &proc[NPROC]; p++) {
16         if (p != myproc()) {
17             acquire(&p->lock);
18             if (p->state == SLEEPING &&
19                 p->chan == chan) {
20                 p->state = RUNNABLE;
21             }
22             release(&p->lock);
23         }
24     }
25 }

```

コード 3: sleep と wakeup の定義

(b) スピンロック mtxp および mtxc は何のためにあるのか。その役割を説明せよ。

(c) コード 2 の 19 行目と 36 行目にある `assert(c)`; は、条件式 c を実行時に検査し、真でなければエラーとなる文である。デバッグやテストの時に、本来ならば必ず成立すべき条件を確認するために用いる。

いま生産者のスレッドと消費者のスレッドがそれぞれ 3 個ずつあり、 N は $N \geq 5$ をみたす整数であるとする。以下の (A)~(J) のうち、コード 2 の 19 行目と 36 行目の `assert` の条件 `p` に用いた時にエラーとならないものをすべて選べ。ただし整数演算のオーバーフローは起こらないものとする。

- (A) $ic \geq ip$
- (B) $ic \leq ip$
- (C) $nf.count + ne.count == N$
- (D) $ic \leq ip + 5$
- (E) $0 \leq ip - ic \ \&\& \ ip - ic \leq N - 5$

(F) $0 \leq ip - ic \ \&\& \ ip - ic \leq N$

(G) $0 \leq ip + ic \ \&\& \ ip + ic \leq N + 5$

(H) $N - nf.count == ip \% N$

(I) $ne.count == ip - ic$

(J) $N - 5 \leq ip + ic \ \&\& \ ip + ic \leq N + 5$

(d) コード 1 の 8~10 行目を以下のように変更すると、`s->count` の値が 0 より小さくなってしまうことがある。その理由を説明せよ。

```

8     if (s->count == 0) {
9         sleep(s, &s->lock);
10    }

```

2. xv6 が動作する RISC-V プロセッサでは、アドレスは 64 ビット中の下位 39 ビットが用いられている。ページテーブルは 3 段で、各ページテーブルのインデックスは 9 ビットである。

xv6 で以下を含むプログラムを実行したところ、2 行目の `printf` の呼び出しによって `0x0000000000003010` と表示された²。

```

char *p = malloc(1024 * 1024 * 4);
printf("%p\n", p);

```

(a) 1 ページの大きさ (バイト数) を答えよ。

(b) 1 行目の `malloc` によって確保され、ユーザプログラム中で使用可能になるメモリ領域の大きさ (バイト数) を答えよ。

(c) 1 行目の `malloc` によって確保されるメモリ領域の割り当てに必要なページ数を答えよ。ただし xv6 の `malloc` の実装では、使用可能になるメモリ領域の前に確保した領域の大きさを記録するためのヘッダ領域を必要とする。上記の場合、実は 16 バイトのヘッダが付随しており、`malloc` の返回值から 16 を引いたアドレスがページ境界になっている。

(d) 1 行目の `malloc` によって確保されるメモリ領域を、物理メモリに割り当てるために使用されるページテーブルエントリの数を答えよ。

²`printf` における `%p` はポインタの値を 16 進数で表示する

(e) malloc の実装に用いられている, メモリ領域の確保のためのシステムコールあるいはライブラリ関数を以下から一つ選べ.

```
pipe  mknod  sbrk  fstat  wait  exit
dup   write  read  open  link  exec
```

3. xv6 のファイルシステムにおいて, inode ブロックに格納される dinode 構造体は以下のように定義されている.

```
struct dinode {
    short type;           // ファイルタイプ
    short major;          // 主デバイス番号
    short minor;          // 副デバイス番号
    short nlink;           // リンク数
    uint size;             // ファイルサイズ
    uint addrs[NDIRECT+1]; // ブロック参照
};
```

マクロ NDIRECT は 12 と定義されている. addrs[0] から addrs[NDIRECT-1] の 12 個がデータブロックへの直接参照で, addrs[NDIRECT] が間接参照である. ブロック番号を表す uint 型は 4 バイト (32 ビット) である. またブロックサイズは 1024 バイトである.

(a) 25600 バイトのファイルが占めるデータブロックの数はいくつか. 間接参照ブロックが必要な場合はそれも数えること. i-node, ビットマップ, ログのためのブロックは数えなくてもよい.

(b) dinode 構造体のフィールド nlink の値は, 当該構造体が表すファイルがディレクトリから参照されている数を表す. ここで xv6 において以下のようなコマンドを実行したとする (\$ はシェルのプロンプト). 実行後のファイル foo/hello.txt およびディレクトリ foo を表す dinode 構造体の nlink の値をそれぞれ記せ.

```
$ mkdir foo
$ echo Hello > foo/hello.txt
$ ln foo/hello.txt foo/salute.txt
$ mkdir foo/bar
$ cat foo/salute.txt > foo/bar/hi.txt
```

(c) nlink の値が正しい値と異なる場合に起こりうる不具合を挙げよ.